Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

Clase grabada

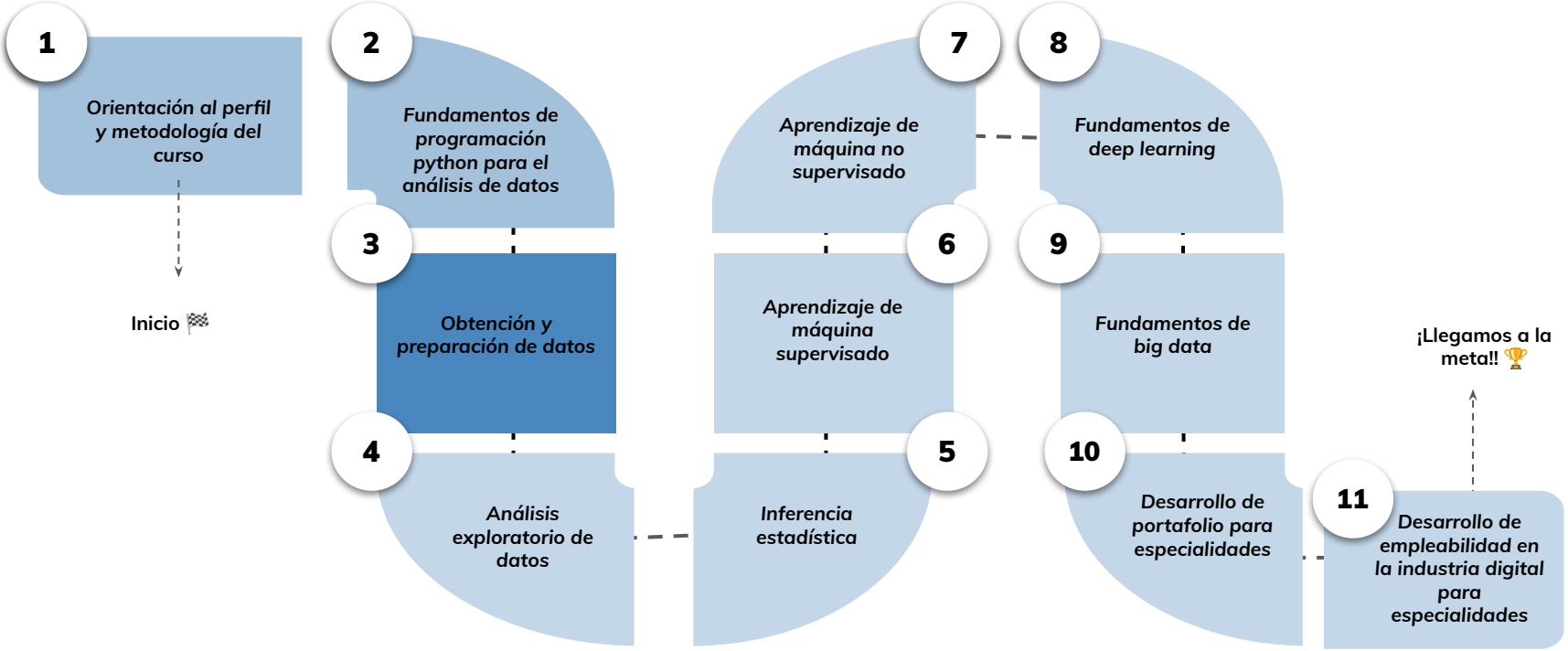
› Agrupamiento y pivoteo de datos - Parte I

Aprendizaje Esperado 6: Aplicar técnicas de unión, agrupamiento y pivoteo de datos utilizando la librería pandas para el reacomodo de la información para la resolución de un problema.

alkemy

Hoja de ruta

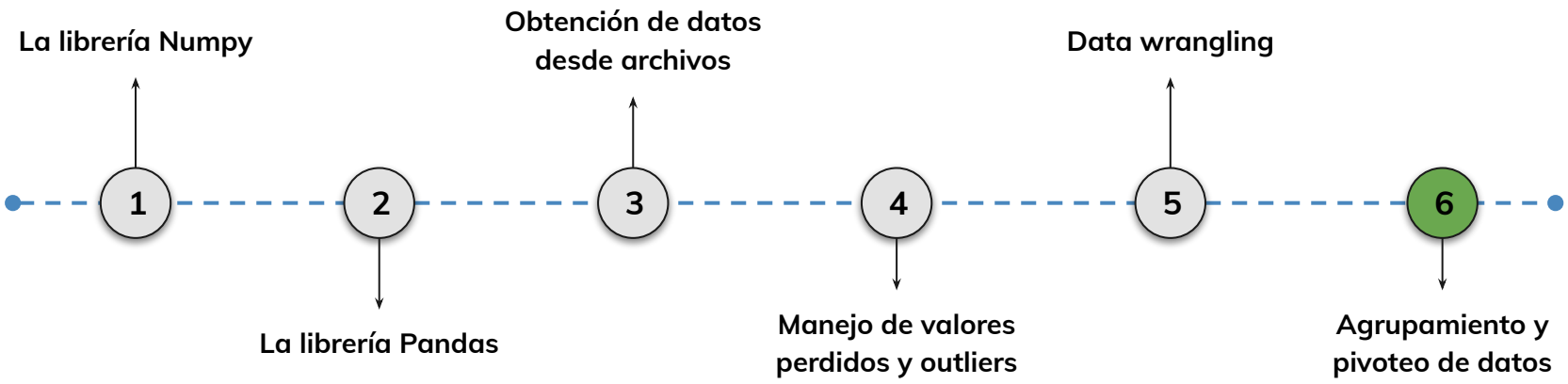
¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



alkemy

Roadmap de lecciones

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



alkemy

Learning Path

¿Cuáles temas trabajaremos hoy?

6.

Agrupamiento y resumen de datos

Aprenderás a utilizar agrupamientos y funciones de agregación para resumir y analizar grandes volúmenes de datos. Dominarás groupby, funciones estadísticas combinadas y creación de tablas dinámicas con Pandas.

- **Agrupación de datos**
  - groupby() por una o varias columnas
  - Funciones como sum, mean, count
- **Agregación múltiple y pivoteo**
  - Uso de .agg()
  - Creación de .pivot_table()

alkemy

Objetivos de aprendizaje

¿Qué aprenderás?

- Aprender a agrupar datos por categorías.
- Aplicar funciones de agregación.
- Transformar estructuras con .groupby() y .pivot_table().

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos 📚:

→ *Conversión y reestructuración de DataFrames.*

→ *Manejo de índices.*

→ *Renombramiento y limpieza avanzada.*

alkemy

Agrupamiento y pivoteo de datos

AGRUPAMIENTO Y PIVOTEO DE DATOS

En esta sección exploraremos las técnicas fundamentales para reorganizar y estructurar datos utilizando Pandas, enfocándonos en el agrupamiento con indexación jerárquica, la función groupby, y los métodos de pivoteo y despivoteo de DataFrames.

Agrupamiento con indexación jerárquica

La indexación jerárquica en Pandas permite estructurar datos en múltiples niveles, facilitando su organización y segmentación. Es particularmente útil en conjuntos de datos con múltiples categorías relacionadas, ya que permite agrupar información sin necesidad de duplicar valores en columnas adicionales.

El uso de índices jerárquicos ayuda a representar datos de manera más compacta y estructurada, proporcionando mayor flexibilidad en la manipulación y el acceso a la información.

alkemy

```python
import pandas as pd

# Crear un DataFrame
datos = {'Región': ['Norte', 'Norte', 'Sur', 'Sur'],
         'Ciudad': ['A', 'B', 'A', 'B'],
         'Ventas': [100, 150, 200, 250]}

df = pd.DataFrame(datos)

# Definir un índice jerárquico
df.set_index(['Región', 'Ciudad'], inplace=True)
print(df)
```

Ejemplo de creación de un índice jerárquico

En este ejemplo se muestra cómo crear un DataFrame con un índice jerárquico de dos niveles, utilizando las columnas 'Año' y 'Trimestre' como índices. Esto permite organizar los datos de ventas de manera estructurada por períodos de tiempo.

```python
print(df.loc['Norte'])
```

Acceso a datos con índice jerárquico

Esto permite acceder a los datos de manera estructurada, facilitando la selección de subconjuntos específicos mediante los diferentes niveles del índice.

Ventajas de la indexación jerárquica

Segmentación multinivel

Facilita la segmentación de datos en múltiples niveles, permitiendo análisis más detallados.

Acceso eficiente

Permite acceder a subconjuntos específicos de manera eficiente mediante la selección por nivel.

Mejor organización

Mejora la organización y representación de datos en análisis tabulares.

Reducción de redundancia

Reduce la redundancia en columnas, ya que se pueden almacenar múltiples niveles dentro del índice.

Agrupamiento de datos con la función groupby

El método groupby() en Pandas permite dividir un conjunto de datos en grupos basados en una o más columnas y aplicar funciones agregadas a cada grupo. Este método es clave en la generación de estadísticas resumidas y en el análisis exploratorio de datos categorizados.

alkemy

Ejemplo de uso de groupby()

El uso de groupby() permite realizar análisis agregados sobre diferentes categorías en un conjunto de datos, facilitando la obtención de estadísticas clave. En este ejemplo, se agrupan los datos por la columna 'Categoría' y se calcula el promedio de los valores en la columna 'Ventas'.

```
# Crear un DataFrame
df = pd.DataFrame({'Departamento': ['Ventas', 'Ventas', 'TI', 'TI'],
                   'Empleado': ['Ana', 'Luis', 'Carlos', 'Sofía'],
                   'Salario': [50000, 54000, 60000, 62000]})

# Agrupar por Departamento y calcular el salario promedio
df_grouped = df.groupby('Departamento')['Salario'].mean()
print(df_grouped)
```

Funciones agregadas comunes con groupby()

.mean()

Media de los valores de cada grupo.

.sum()

Suma total de los valores por grupo.

.count()

Cantidad de elementos dentro de cada grupo.

.max() y .min()

Valores máximo y mínimo dentro de cada grupo.

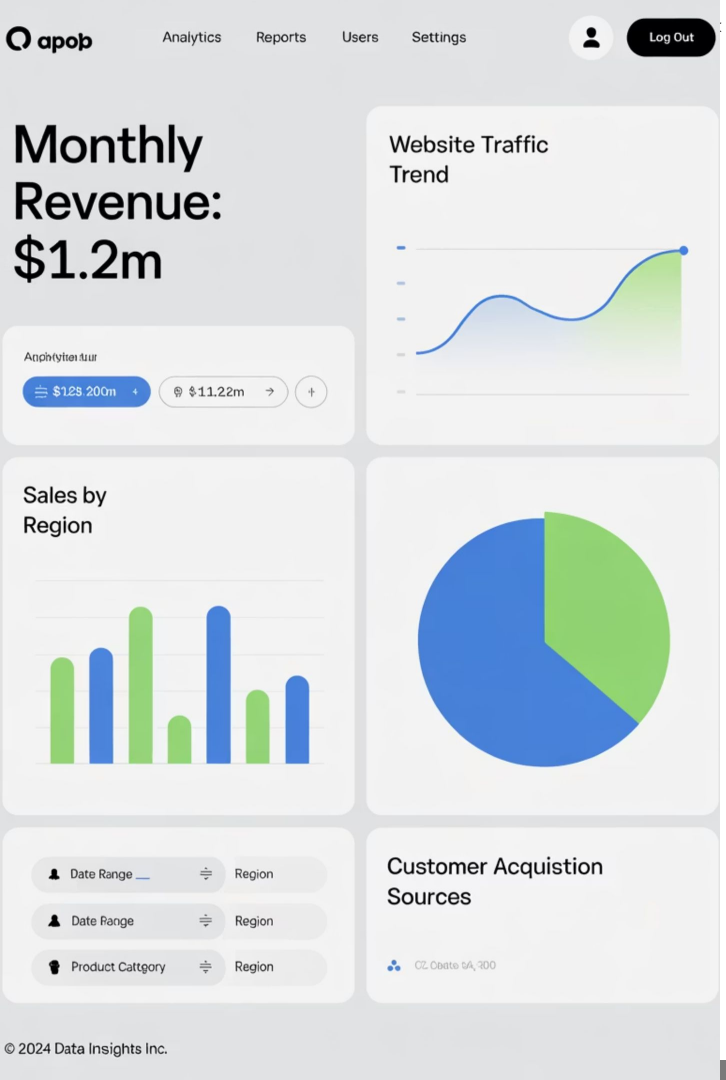
.agg()

Aplicación de múltiples funciones agregadas a la vez.

Ejemplo de uso de .agg() para múltiples funciones

Este método es fundamental en el análisis de grandes volúmenes de datos y permite generar reportes con métricas clave para la toma de decisiones basada en datos. En el ejemplo, se aplican múltiples funciones agregadas (suma, media, máximo y mínimo) a diferentes columnas del DataFrame agrupado.

```
# Aplicar múltiples funciones en el agrupamiento
df_grouped = df.groupby('Departamento').agg({'Salario': ['mean', 'max', 'min']})
print(df_grouped)
```



Importancia de groupby() en el análisis de datos

Análisis segmentado

Permite analizar comportamientos específicos por categorías, regiones, períodos de tiempo u otras variables de interés.

Generación de reportes

Facilita la creación de informes resumidos con métricas clave para diferentes segmentos del negocio.

Identificación de patrones

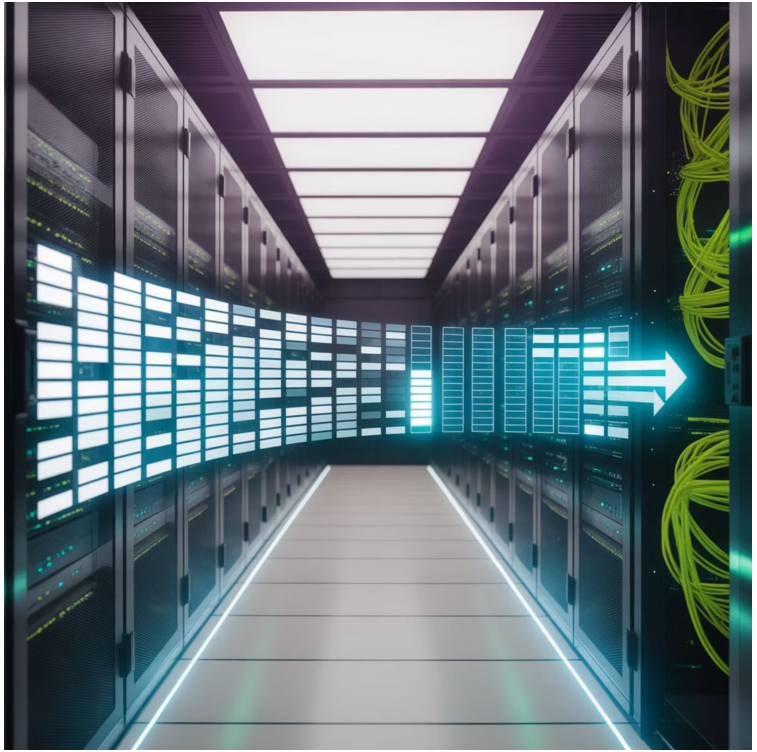
Ayuda a descubrir tendencias y anomalías en subconjuntos específicos de datos.

alkemy

Pivoteo de dataframes

El pivotado de datos convierte datos de formato largo a formato ancho, reorganizando los valores en nuevas columnas. Esta técnica permite transformar un conjunto de datos en el que los valores están organizados en filas en un formato más estructurado, facilitando su análisis y visualización.

El método pivot() de Pandas es una herramienta clave en este proceso. Se utiliza para reorganizar los datos en función de una o más columnas, creando una nueva estructura en la que los valores de una columna específica se convierten en encabezados de columnas.



Ejemplo de pivotado de datos

En este ejemplo, se utiliza el método pivot() para transformar un DataFrame de formato largo a formato ancho. Los valores de la columna 'Categoría' se convierten en nuevas columnas, y los valores de 'Ventas' se distribuyen en la tabla resultante según las combinaciones de 'Fecha' y 'Categoría'.

```
import pandas as pd

# Crear un DataFrame
df = pd.DataFrame({'Fecha': ['2023-01', '2023-01', '2023-02', '2023-02'],
                   'Ciudad': ['A', 'B', 'A', 'B'],
                   'Ventas': [100, 150, 200, 250]})

# Pivotar el DataFrame
df_pivotado = df.pivot(index='Fecha', columns='Ciudad', values='Ventas')
print(df_pivotado)
```

Beneficios del pivotado de datos

1

Visualización intuitiva

Permite visualizar la información en un formato tabular más intuitivo, facilitando la comprensión de relaciones entre variables.

2

Comparación de métricas

Facilita la comparación de métricas a lo largo del tiempo o entre categorías, mejorando el análisis comparativo.

3

Reportes dinámicos

Es útil en la creación de reportes dinámicos y paneles de control que requieren una estructura específica de datos.

4

Análisis exploratorio

Optimiza la manipulación de datos en análisis exploratorios, permitiendo identificar patrones más fácilmente.

Consideraciones al usar pivot()

Valores duplicados

El pivotado requiere que el conjunto de datos no contenga valores duplicados en las columnas utilizadas para el índice y los encabezados. Si existen duplicados, se debe utilizar pivot_table() en su lugar.

Valores nulos

El pivotado puede generar valores nulos cuando no existen ciertas combinaciones de categorías en los datos originales.

Rendimiento

En conjuntos de datos muy grandes, el pivotado puede consumir recursos significativos de memoria y procesamiento.

Despivoteo de dataframes (método melt)

El proceso inverso al pivotado es el despivotado o melt, que transforma un DataFrame de formato ancho a formato largo. Esto es útil cuando se requiere normalizar datos para análisis o cuando un formato tabular no es adecuado para ciertas visualizaciones o modelos analíticos.

El método melt() en Pandas permite convertir un DataFrame en formato ancho a un formato largo, facilitando su análisis y manipulación.

Ejemplo de uso de melt()

En este ejemplo, se utiliza el método melt() para transformar un DataFrame de formato ancho a formato largo. Las columnas de categorías se convierten en valores de una nueva columna 'variable', y sus valores correspondientes se almacenan en la columna 'value'.

```
# Despivotar el DataFrame
df_melt = df_pivotado.reset_index().melt(id_vars='Fecha', var_name='Ciudad',
                                         value_name='Ventas')
print(df_melt)
```

Ventajas del despivotado

Manejo de múltiples categorías

Facilita el manejo de datos cuando se trabaja con múltiples categorías, simplificando la estructura para análisis posteriores.

Compatibilidad con herramientas

Permite compatibilizar los datos con herramientas de visualización y machine learning que requieren formato largo.

Transformación para bases de datos

Es útil en la transformación de datos para bases de datos relacionales, facilitando la normalización.

Flexibilidad de análisis

Mejora la flexibilidad de análisis al permitir realizar filtros y agregaciones dinámicas sobre categorías.

Aplicaciones prácticas del despivotado

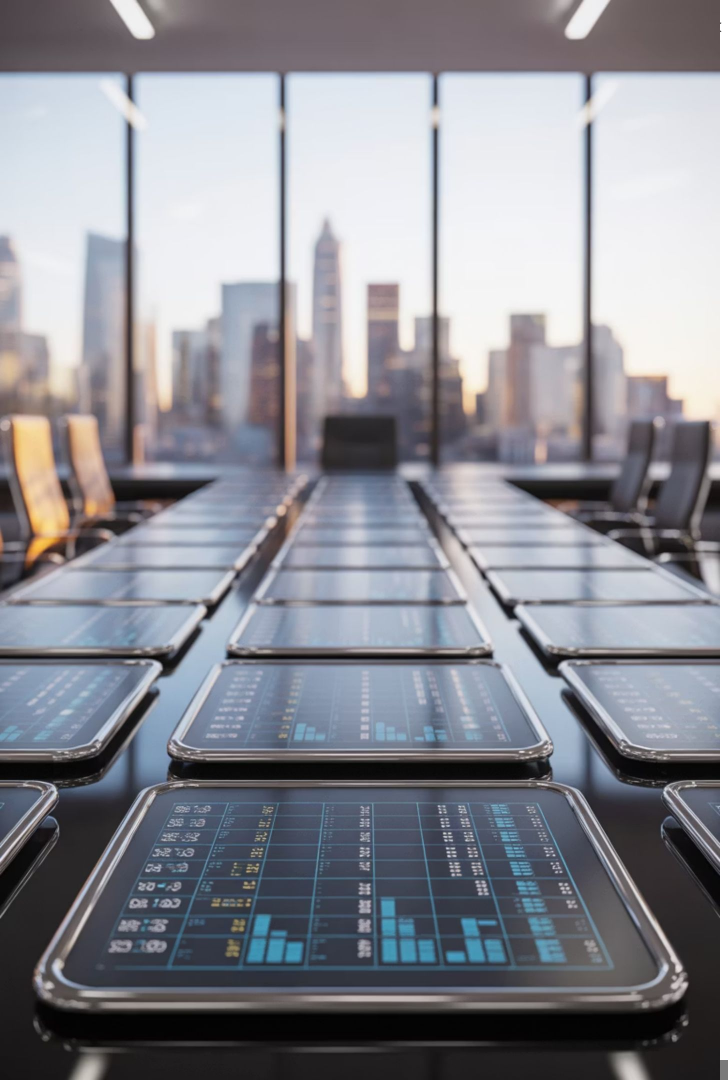
Preparación para visualizaciones

Muchas bibliotecas de visualización como Seaborn y Matplotlib trabajan mejor con datos en formato largo.

Análisis estadístico

Facilita la aplicación de pruebas estadísticas que requieren datos en formato específico.

El formato largo permite crear visualizaciones más flexibles y dinámicas.



COMBINACIÓN Y MERGE DE DATOS

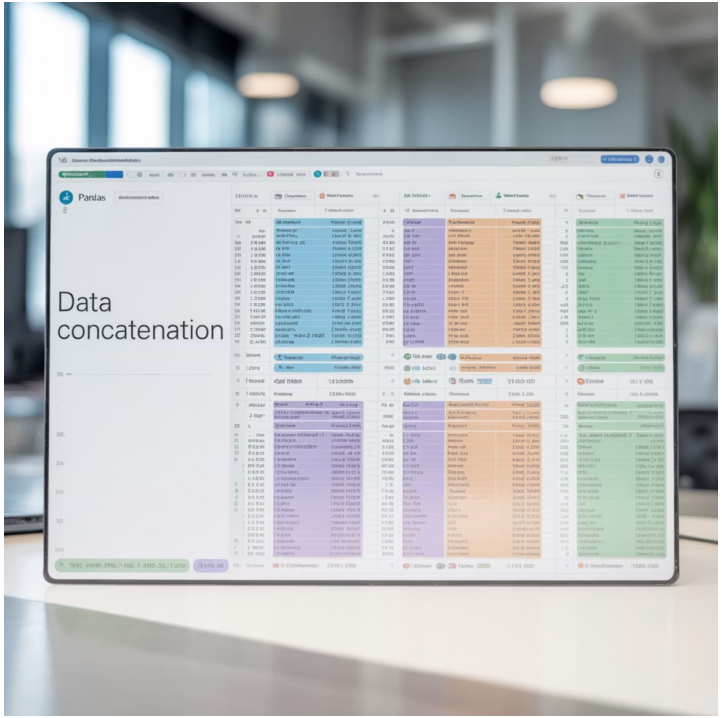
En esta sección exploraremos las técnicas para combinar múltiples conjuntos de datos, enfocándonos en la concatenación de DataFrames y el uso del método merge() para unir datos basados en claves comunes.

alkemy

Concatenación de dataframes

La concatenación de DataFrames permite unir múltiples conjuntos de datos a lo largo de filas o columnas. Es útil cuando se necesita agregar nuevos registros o combinar diferentes fuentes de datos con una estructura similar.

El método concat() en Pandas facilita la combinación de DataFrames, permitiendo especificar el eje de concatenación y manejar posibles valores duplicados.



Ejemplo de concatenación de DataFrames

En este ejemplo, se concatenan dos DataFrames (df1 y df2) a lo largo del eje 0 (filas), creando un nuevo DataFrame que contiene todas las filas de ambos conjuntos de datos. El parámetro ignore_index=True reinicia los índices para evitar duplicados.

```
# Crear dos DataFrames
df1 = pd.DataFrame({'ID': [1, 2, 3], 'Valor': [100, 200, 300]})
df2 = pd.DataFrame({'ID': [4, 5, 6], 'Valor': [400, 500, 600]})

# Concatenar los DataFrames
df_concatenado = pd.concat([df1, df2])
print(df_concatenado)
```

Ventajas de la concatenación

Combinación sin relaciones explícitas

Permite combinar datos sin necesidad de relaciones explícitas entre los conjuntos, simplificando la integración.

Unión de estructuras similares

Es útil para unir conjuntos de datos de la misma estructura, como datos históricos o de diferentes períodos.

Integración rápida

Facilita la integración de múltiples fuentes de datos de manera rápida y sencilla, optimizando el flujo de trabajo.

Opciones de concatenación

Concatenación por filas (axis=0)

Une los DataFrames verticalmente, añadiendo las filas del segundo DataFrame debajo del primero. Es la opción predeterminada.

Concatenación por columnas (axis=1)

Une los DataFrames horizontalmente, añadiendo las columnas del segundo DataFrame a la derecha del primero.

Parámetros útiles

- **ignore_index**: Reinicia los índices para evitar duplicados.
- **join**: Especifica cómo manejar columnas no coincidentes ('inner' o 'outer').
- **keys**: Añade un nivel jerárquico al índice para identificar la fuente de cada fila.

Aplicaciones de la concatenación en ETL

La concatenación se utiliza frecuentemente en procesos de ETL (Extracción, Transformación y Carga) para consolidar datos antes de su análisis.

Extracción de datos

Obtención de datos de múltiples fuentes o archivos.

Concatenación

Unión de los conjuntos de datos limpios en un único DataFrame.

Transformación

Limpieza y preparación de cada conjunto de datos.

Análisis

Procesamiento y análisis del conjunto de datos consolidado.

Live Coding

¿En qué consistirá la Demo?

Resumen de ventas por categoría y región

1. *Agrupar por “categoría” y “región”.*
2. *Calcular suma y promedio de ventas.*
3. *Crear tabla dinámica para mostrar totales.*

Tiempo: 25 minutos

#Momentode Preguntas...

¿ ?

¿Qué hace .groupby() en Pandas?

¿ ?

¿Para qué sirve .agg()?

¿ ?

¿Qué funciones de agregación existen en Pandas?

¿ ?

¿Qué diferencia hay entre groupby y pivot_table?

Momento:

Time-out!

⏰5 -10 min.

alkemy

Ejercicio N° 1

Informe resumido de productos

Informe resumido de productos

Consigna: ✍️

1. Agrupar un dataset de productos por “tipo”.
2. Mostrar cantidad de productos y precio promedio.
3. Aplicar .agg() con dos funciones combinadas.
4. Usar .pivot_table() para ver totales por marca y tipo.

Tiempo 🕐**:** *30 minutos*

¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ ***Agrupaste datos de acuerdo a diferentes criterios.***
- ✓ ***Aplicaste múltiples funciones de agregación.***
- ✓ ***Dominas el uso de .agg() para resumen múltiple.***
- ✓ ***Usaste .pivot_table() para generar resúmenes claros.***

¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las ***actividades disponibles en la plataforma.*** *Estas propuestas son clave para afianzar lo trabajado y* ***forman parte obligatoria del recorrido de aprendizaje.***

💪 Análisis de caso ——— 💪 Selección Múltiple

💪 Comprensión lectora

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

alkemy

< ¡Muchas gracias! >

alkemy